EXERCICE 08 · CODE COMPLET

Formulaire d'enregistrement

Tous les fichiers utilisés, en entier, dans l'ordre de lecture.

1. **correction** (3 fichiers)

1 correction

Styles fournis (copiés tels quels, pas tapés en live) : src/App.css (195 lignes)

Fichiers générés par Vite, non modifiés (non affichés) : index.html, vite.config.js, .gitignore, eslint.config.js, public/vite.svg

package.json (gabarit Vite) : dépendances ajoutées : @hookform/resolvers ^5.2.2, react-hook-form ^7.68.0, zod ^4.2.1

FICHIER **correction/src/main.jsx** 6 lignes

```jsx
import { createRoot } from "react-dom/client";
import App from "./App.jsx";

createRoot(document.getElementById("root")).render(
  <App />
);
```

FICHIER **correction/src/App.jsx** 8 lignes

```jsx
import "./App.css";
import Form from "./components/Form";

function App() {
  return <Form />;
}

export default App;
```

FICHIER **correction/src/components/Form.jsx** 113 lignes

```jsx
// Exercice 8 · correction : le template + la validation avec React Hook Form et Zod
import { useForm } from "react-hook-form";
import { z } from "zod";
import { zodResolver } from "@hookform/resolvers/zod";

const schema = z.object({
  // Champ "Nom complet"
  fullName: z
    .string()
    .min(3, "Min 3 caractères")
    .max(50, "Max 50 caractères")
    .regex(/^[A-Za-zÀ-ÿ\s-]+$/, "Lettres uniquement"),
  email: z.string().email("Email invalide"),
  // Champ supplémentaire "Experience"
  experience: z
    .number()
    .int("Nombre entier uniquement")
    .min(0, "Min = 0")
    .max(10, "Max = 10"),
  // Checkbox "Terms & Conditions"
  terms: z
    .boolean()
    .refine(
      (v) => v === true,
      "Vous devez accepter les Conditions d'utilisation"
    ),
```

```
});

function Form() {
  const {
    register,
    handleSubmit,
    formState: { errors, isSubmitSuccessful, isValid },
  } = useForm({
    resolver: zodResolver(schema),
    defaultValues: { fullName: "", email: "", experience: 0, terms: false },
    mode: "onSubmit",
  });

  const onSubmit = (data) => {
    console.log("Send to server:", data); // ici tu fais ton appel API
  };

  return (
    <div className="container">
      <h1>Register Component</h1>
      <h2>Enter your information</h2>

      {isSubmitSuccessful ? (
        <div className="success">Enregistrement OK</div>
      ) : (
        <form name="login-form" onSubmit={handleSubmit(onSubmit)}>
          <div className="form">
            <div className="form-group">
              <label>Full name</label>
              <input type="text" {...register("fullName")} />
              {errors.fullName && (
                <p className="error">{errors.fullName.message}</p>
              )}
            </div>

            <div className="form-group">
              <label>Email</label>
              <input type="text" {...register("email")} />
              {errors.email && (
                <p className="error">{errors.email.message}</p>
              )}
            </div>

            <div className="form-group">
              <label>Experience</label>
              <input
                type="number" // Changé de "text" à "number"
                {...register("experience", { valueAsNumber: true })} // Ajouté valueAsNumber
              />
              {errors.experience && (
                <p className="error">{errors.experience.message}</p>
              )}
            </div>

            {/* Terms & Conditions */}
            <div className="form-group">
              <label className="checkbox">
                <input type="checkbox" {...register("terms")} />
                <span>
                  J'accepte les{" "}
                  <a href="#" target="_blank" rel="noreferrer">
                    Conditions d'utilisation
                  </a>
                </span>
              </label>
```

```
              {errors.terms && (
                <p className="error">{errors.terms.message}</p>
              )}
            </div>

            {Object.keys(errors).length > 0 && (
              <p className="error">Veuillez corriger les champs invalides.</p>
            )}

            {isValid && <p className="success">Form valid (ready to send)</p>}

            <button className="btn" type="submit">
              Register
            </button>
          </div>
        </form>
      )}
    </div>
  );
}

export default Form;
```